Interview & Concept Guide
Bank Management System ■

Designed for Placement Preparation at Cognizant, TCS, Accenture, Wipro, Infosys, and Deloitte

> ***The 60-Second Project Pitch (To use when asked: 'Tell me about your project'):***
> *"I built a production-quality **Bank Management System** in Core Java using a **decoupled layered architecture** (Controller-Service-Repository pattern). It simulates a banking system with role-based logins (Admin and Customer), transactional security, auto-generated account numbers, and transaction logs. Instead of a database, I implemented a robust, synchronized flat-file database handler using Java File I/O. The project is designed strictly following **SOLID principles** and core **Object-Oriented Programming (OOP)** patterns. I used modern Java features like the **Streams API, Lambdas, Optionals, and the Java Time API** to make the code clean, thread-safe, and highly maintainable."*

■■ 1. Layered Architecture (De-coupling)

This project is structured into 5 distinct layers to enforce separation of concerns, ensuring high decoupling and testability:

- **Presentation Layer (UI):** Displays console menus and handles scanner inputs (ConsoleMenu).
- **Controller Layer:** Intercepts incoming requests and handles presentation exception mappings (BankController).
- **Service Layer:** Validates business constraints and executes bank transaction rules (BankServiceImpl).
- **Repository Layer:** Manages read/write streams to persist data (AccountRepositoryImpl).
- **Storage Layer:** Local flat files (accounts.txt, transactions.txt, admins.txt) acting as our database.

Interviewer value: Explain that if we switch the presentation layer to a React UI or the storage layer to MySQL tomorrow, the core business rules layer (Service) remains completely untouched because they communicate only via interfaces.

■ 2. Core OOP Principles Applied

A. Encapsulation

Wrapping data (variables) and code (methods) together. In our models (User.java and Account.java), all variables are declared private. Access is strictly controlled through public getters, setters, and validation methods (like withdraw).

B. Inheritance

Subclassing parent attributes to promote code reusability. Our project implements:
• User (Abstract class) → Extended by Customer and Admin.
• Account (Abstract class) → Extended by SavingsAccount and CurrentAccount.

C. Polymorphism

The ability of an object to take on many forms. Our project uses **runtime polymorphism** (dynamic method dispatch). The abstract class Account defines the method public abstract double getMinBalance();. At runtime, Java dynamically calls the correct overridden version—returning 500.0 for a SavingsAccount or 2000.0 for a CurrentAccount.

D. Abstraction

Hiding execution details. Callers interact strictly through interfaces like BankService and AccountRepository, knowing nothing of file paths or file buffer operations.

■ 3. SOLID Principles Implemented

Principle	Concept	How Implemented in Project
Single Responsibility (SRP)	A class should do exactly one thing.	ConsoleMenu only handles console views. ValidationUtil checks regex. FileUtil handles IO operations. BankServiceImpl executes core transactions.
Open/Closed (OCP)	Open for extension, closed for modification.	To add a new account type (e.g. JointAccount), we simply subclass Account without modifying existing code.
Liskov Substitution (LSP)	Subclasses must substitute parent classes cleanly.	We can pass SavingsAccount and CurrentAccount objects anywhere an Account instance is expected without breaking functionality.
Interface Segregation (ISP)	Interfaces should be focused and split.	BankService and AccountRepository expose precise methods required by callers.
Dependency Inversion (DIP)	Depend on abstractions, not concretions.	BankServiceImpl depends on the AccountRepository interface. This makes it trivial to swap file-persistence with database persistence.

■ 4. Modern Java 8+ Features Used

- **Streams API & Lambdas:** Utilized for functional-style collections queries. For example, filtering matches in searchCustomers(query):

```
repository.getAllCustomers().stream().filter(c ->
c.getName().contains(searchKey)).collect(...)
```

- **Optional Wrapper:** Helps indicate the potential absence of customer entries safely (e.g. Optional<Customer>) and prevents NullPointerException.
- **Java Time API (LocalDate & LocalDateTime):** Replaces outdated, thread-unsafe classes like java.util.Date with immutable calendar APIs.
- **Method References:** Replaces verbose lambdas with cleaner method calls (e.g., User::getName).

■ 5. File Handling & Concurrency

- **BufferedReader & BufferedWriter:** Wrapped around standard file streams to perform optimized buffered character operations.
- **Flat File Serialization:** Objects are serialized to pipe-delimited CSV formats on state-changes, and loaded on boot, simulating database operations.

- **Thread Safety:** Shared generation operations (like AccountNumberGenerator.generate()) are declared synchronized to prevent lock/race-condition conflicts.

■ 6. Common Interview Questions & Answers

Q: Why did you use File Handling instead of a SQL Database?
A: *"I chose File Handling to demonstrate my core knowledge of Java I/O streams and manual serialization. However, because the application is decoupled through the AccountRepository interface, we can shift to a relational database (SQLite/MySQL) using JDBC by writing a new implementation class without changing a single line of business logic."*

Q: How did you implement transactional safety during transfers?
A: *"The transfer method in BankServiceImpl acts atomically. It validates that both sender and receiver accounts exist, checks available balances, and performs the deduction and credit together. If any validation fails, a checked exception is thrown before any file writes are executed, simulating a database rollback."*

Q: How does the system handle security locks?
A: *"The Account model has a failedAttempts counter and a locked flag. Upon 3 consecutive failed attempts during login, the account is set to locked, and details are written to accounts.txt. Locked users are denied entry and can only be unlocked by an administrator."*